

ORIGINAL RESEARCH

HybridDep: An elastic hybrid resources allocation strategy for I/O-intensive applications

Pengmiao Li^{1,2}  | Yuchao Zhang² | Shaoxuan Yun² | Fucai Yu¹ | Aizhi Wu¹

¹Urban Operation Safety Research Center, Beijing Academy of Emergency Management Science and Technology, Beijing, China

²State key Laboratory of Networking and Switching Technology, Beijing University of Posts and Telecommunications, Beijing, China

Correspondence

Yuchao Zhang, Beijing University of Posts and Telecommunications, Beijing, 100080, China.
Email: yczhang@bupt.edu.cn

Funding information

Beijing Nova Program, Grant/Award Number: 2023140; National Natural Science Foundation of China, Grant/Award Number: 62172054; Beijing Science and Technology Planning Project, Grant/Award Number: Z231100003823005

Abstract

Along with the rapid development of B5G/6G, the number of applications grows rapidly and the data amount explodes exponentially, putting a massive burden on the resource-limited edge servers. To fully utilize the limited resources, virtualization technology is introduced to provide elastic deployment for applications in edge servers. But for I/O-intensive applications, allocating elastic resources is not as easy as for compute-intensive ones, because the amount of required I/O resources is unknown due to the request uncertainty. Many existing researches try to solve this multi-application deployment problem by peaks clipping and valleys filling, to resource utilization. However, in fact, the times of peaks and valleys of most hybrid deployed applications are similar to each other, which invalidates those traditional solutions. To address this challenge, the actual data is analysed and complementary peak and valley periods in time and space dimensions are found. Based on this finding, an elastic hybrid deployment strategy *HybridDep* is proposed, for multiple I/O-intensive applications. Validated by simulation experiments using real datasets and traces, this algorithm can reduce about 3.2% deployment cost than the compared algorithm.

1 | INTRODUCTION

With the development of B5G/6G and AI technology, the number of applications and users is increasing exponentially. According to the report [1], nearly 300 million mobile applications will be downloaded by 2023. The number of worldwide social media users is 5.31 billion, which is more than 67.1% of the world's population [2]. These vast applications and users generate much content, introducing high storage and bandwidth resource requirements on edge servers (ESs). However, each ES's storage and bandwidth are quite limited, making it impossible to carry so many applications simultaneously. Therefore, it is crucial to design an elastic application deployment strategy to increase the number of successfully deployed applications and reduce the cost of infrastructure service providers (ISP).

To achieve elastic application deployment, virtualization techniques have been introduced by prior researchers. Based on virtualization technology, an ES can be divided into multiple virtual microservices to provide separate applications.

Such virtualization allows multiple applications to be deployed in a single ES (multiple microservices), leading to

much higher utilization of server resources. These methods work quite well for compute-intensive applications [3–7], computing resources can be allocated once needed and released immediately when the tasks finish. However, in I/O-intensive applications, application latency changes along with the time-varying requests [8, 9], which constantly changes the remaining storage and bandwidth. This means that I/O cannot be scaled up and down precisely, due to unseen request latency. Therefore, how to elastically deploy these I/O-intensive applications to reduce hybrid deployment costs is the main concern.

As each application has peaks and valleys in request volume because of different request frequencies, it is easy to adopt peak load shifting methods to implement complementary multi-application deployment. However, these solutions simplify the real environment and ignore a real situation in reality: each application has similar periods of peaks and valleys, which means that it is impossible to adopt any peak-clipping and valley-filling methods, as shown in Figure 1. The three applications have peaks/valleys simultaneously, so it is impossible to cut one's peak and fill the other's valley. Some solutions [10, 11] that employ machine learning to predict resource load for

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2025 The Author(s). *IET Communications* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

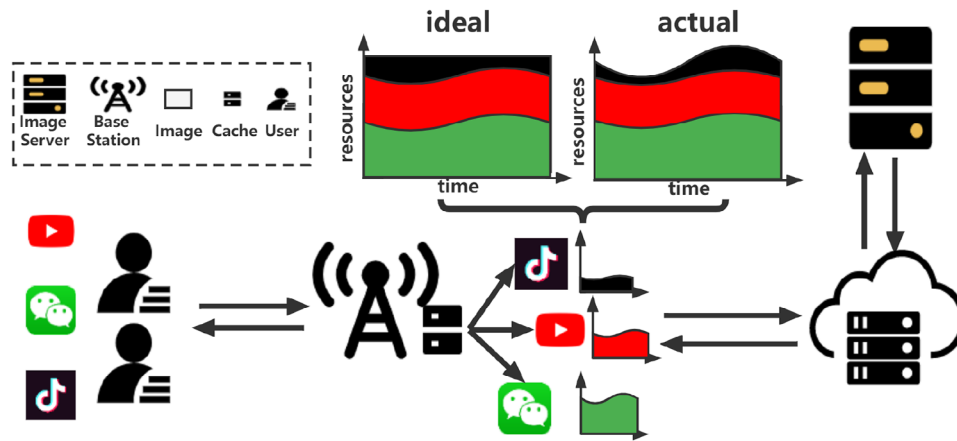


FIGURE 1 The ideal and actual resources allocated based on temporal dimension.

resource deployment consume a significant amount of time and resources during the model training and prediction phases. Consequently, they are not suitable for deployment on edge servers with limited resources.

To address this challenge, we delve deeper into a vast amount of real application data and discover an opportunity for staggered request patterns among different applications, analysed from both spatial and temporal dimensions. Based on this, we propose an elastic hybrid resources allocation strategy *HybridDep*, which consists of two phases: coarse-grained scheduling and fine-grained tuning. Coarse-grained minimizes deployment costs based on the opportunity by integer linear programming. Fine-grained dynamically adjusts each application's resources allocated to meet the time-varying application resource requirements. Finally, we implement *HybridDep*, and the results show that the deployment cost is 3.2% below the *Optimal's* while ensuring the high quality of service (QoS).

The main contributions of this paper are listed as follows:

- Identifying the peak load shifting method is unsuitable in real applications: It is difficult to know the amount and time of increasing and decreasing resources for I/O-intensive applications than compute-intensive ones. Therefore, how to implement the elastic hybrid allocation of these resources for I/O-intensive applications is the paper study problem called *ElaDep*.
- Proposing a hybrid deployment strategy for multiple I/O-intensive applications based on two-dimensional staggered peaks: The challenge is that achieving the peak clipping and valley filling is uneasy due to similar occurrence times about peaks and valleys for each application in the hybrid deployment. By analysing real data sets, the staggering request peaks and valleys of different applications in the same region can bring an opportunity to address the challenge. Then, we propose an elastic hybrid resources allocation strategy *HybridDep* based on this finding to resolve the problem *ElaDep*.
- Implementing prototype deployment: Through experimental verification, our strategy *HybridDep* can reduce the deployment cost by up to 14.6% in 1 h than *Optimal*.

The remainder of the paper is organized as follows: In Section 3, we first propose the problem *ElaDep*, then find a potential opportunity by deeply analysing the real traces, and finally, present the system. We formulate the deployment problem *ElaDep* for I/O-intensive multi-application in Section 4. In Section 5, the design of the proposed algorithm *HybridDep* architecture is presented. Section 6 conducts experiments with a detailed introduction and analysis of the results. Section 2 mentions previously published related works of ES deployment. Conclusions are presented in Section 7.

2 | RELATED WORK

Resource allocation has been a critical concern for ISPs. Excellent or lousy resource deployment directly affects the cost of ISP and the QoS of applications.

Paper [12] investigates the CSQ-link problem and introduces a new metric latency called S2SL. To resolve the problem above, they propose a frequency-aware strategy *Frend* to minimize the request number (the cost of ISPs) of ESSs for a single application while ensuring low S2SL. Paper [13] studies the cached deployment optimization (CaDeOp) problem of determining how many servers, energy, and bandwidth resources to a provision in each cache AS, that is, each AS chosen for cache deployment. Paper [14] offers energy-optimal edge content cache and dissemination designs for hot spots and rural areas, which is exploited Lyapunov optimization and optimal control theory. There are also studies of layered deployments in heterogeneous networks [15, 16]. However, existing deployment strategies are less resilient in I/O-intensive applications due to the time-varying nature of requests, thus affecting resource utilization.

Since CDN (as an edge server) can mainly provide the service for the I/O-intensive application, it generates a mounting waste of resources such as CPU. Therefore, virtual deployment [3–7] is proposed to isolate the CPU from storage for an elastic deployment to supply other compute-intensive-based services. By using virtual CDNs, content providers can choose the area

where they use caching servers based on the geographic demand patterns of their users, which is expected to reduce the cost of CDNs [17]. According to a study on the container image pulling acceleration [18, 19] and micro-service deployment, the service quality of users can be improved [20–22]. Papers [23, 24] consider micro-service placement and request scheduling in a layer-sharing context to maximize edge throughput. Paper [23] proposes a layer-sharing micro-service deployment and image-pulling strategy, which explores the advantage of layer-sharing to speed up micro-service startup and lower image storage consumption. Nevertheless, massive resource wasting by redundancy in layers remains as only a tiny proportion of layers are identical and can be shared. Joint caching and scheduling strategies enable saving some resource consumption on storage and networking [4, 25].

Although existing deployment strategies are elastic for some compute-intensive applications, they are unsuitable for the I/O-intensive ones because the amount of required I/O resources is unknown due to the request uncertainty [8, 9]. Some papers [10, 11], while adopting machine learning to predict resource demands and deploying accordingly, or directly deploying based on resource demands, neglect the time and resource consumption required for algorithm training and prediction processes, which in certain scenarios appears impractical. Therefore, we propose an elastic hybrid resource allocation strategy for multiple I/O-intensive applications to minimize cost without affecting applications' QoS in this paper.

3 | MOTIVATION AND SYSTEM

In this section, the motivation and problem are described in Sections 3.1 and 3.2 shows the system statement of this paper.

3.1 | Motivation and problem

In order to understand the problem and discover the motivation to solve it, we analysed the real data set and came up with the following perspective. It is worth noting that all the graphs in this paper utilize datasets from a CDN company in China, namely *ChuangCache*.

3.1.1 | The peak clipping and valley filling are not easy complements

To fully utilize the limited resources of edge servers, it is necessary to allocate resources elastically for applications by virtualization technology, thus reducing cost. In I/O-intensive applications, the number of resources allocated and the time of releasing surplus ones cannot be known explicitly. This is because the frequency of requests affecting the demand for resources is time-varying and continuous [12] (see Figure 2a). The essence of request data generation is that users use applications. Due to varying lifestyles, users will access the application

at different times. This leads to the application request being generated with time-varying. We can easily find that the request frequency disparity is significant from different periods. The period with high request frequency is called a peak, such as hour 15. The period with low request frequency is called a valley, such as hour 6. Each I/O-intensive application has peak and valley periods in storage and bandwidth resources. The network traffic is a form of direct reflection of the request frequency. It means the storage and bandwidth resources allocation need to change with time to reduce and avoid congestion, reducing the transmission latency. In other words, storage and bandwidth resource allocation also have peaks and valleys periods. Figure 2b presents bandwidth resources of three applications based on request frequency in one day. The horizontal coordinate represents the time (hour), and the vertical coordinate represents the required bandwidth resources ratio. We can find that different applications have similar periods during peaks and valleys. It is not wise if we clip peaks and fill valleys by hybrid multi-application deployment based on time. It is because the peak and valley periods are still generated and shown in Figure 2c. The horizontal coordinate represents the time (hour), and the vertical coordinate represents the total required bandwidth resources ratio of the three applications. It shows that hybrid multi-application deployment based on time ineffectively achieves peak clipping and valley filling. There is a similar phenomenon to bandwidth resources when assigning the right storage resources to the application. This paper ignores introducing this phenomenon about storage resources. Therefore, the peak clipping and valley filling are inaccessible in the elastic deployment for I/O-intensive applications.

3.1.2 | Peaks and valleys are staggered in the same region

In order to solve the above difficulties, it is not feasible to use the traditional way of peak clipping and valley filling in the elastic deployment resources for I/O-intensive applications, we investigated some prior researchers' studies. Through prior researchers' findings, there is a different preference of users for applications in different regions (geographic areas) [26]. It means there may be a significant usage of a particular application and inefficient use of other applications in different regions (geographic areas) in a certain period. Based on this observation, we deeply analyse the actual data of request frequency in detail. This view exists in real-life scenarios, as shown in Figure 2d. The horizontal coordinate represents the region ID, and the vertical coordinate represents the request frequency. For example, in region 2, App1 and App2 are in peak and valley periods, respectively. In region 10, the periods of the two Apps are opposite. So that a complementary phenomenon is obtained to clip peaks and fill valleys. Thus, we can perform the elastic hybrid allocation of bandwidth and storage resources for multiple I/O-intensive applications based on the peak and valley staggered in the same region.

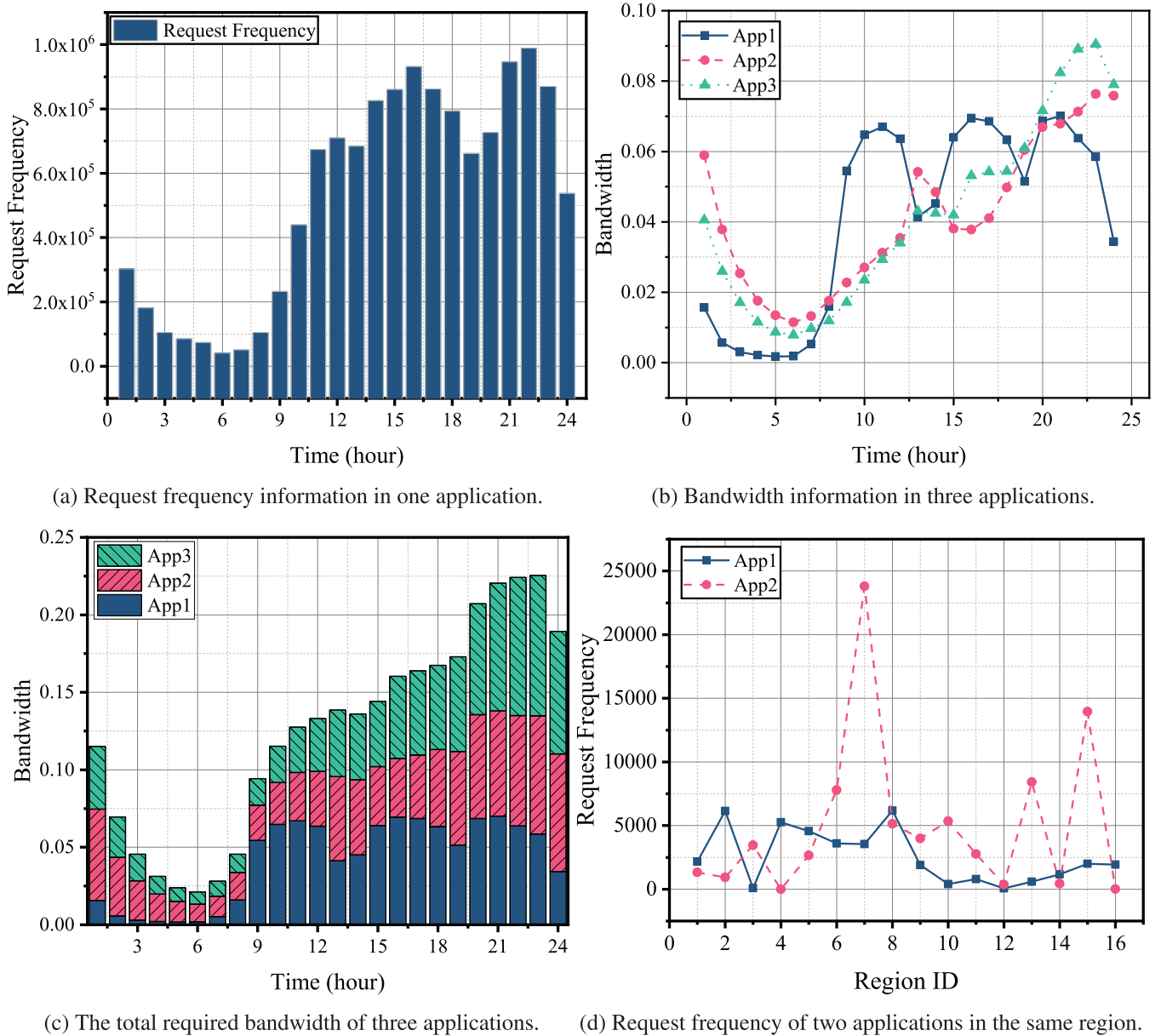


FIGURE 2 (Data analysis) Request and bandwidth information.

3.2 | System statement

We consider an elastic hybrid application deployment system where one ES can provide storage and bandwidth resources for multi-applications. This needs to adopt the virtual technology for splitting one ES into more microservices for different applications. Each image of virtual technology stores some system files, which cost some storage resources. This means that the fewer applications deployed on the same microservice, the better. At runtime, non-local microservice images must be frequently pulled from remote registries to local servers, resulting in large pulling traffic [23]. The microservices generated in one ES are capacitated to reduce pulling time. So, the main latency of this paper is transmitted latency, including client-server latency and server-cloud latency.

A microservice may receive requests from one application and can directly process these requests. If the requested content is cached in the microservice, it will be returned to the user (client), which is called 'hit' and generates the client-server latency. Otherwise, the requested content is uncached in the microservice, called 'miss,' and generates the server-cloud latency and the client-server latency. To avoid significant transmission latency due to end-users and servers being in different regions, we restrict servers from being deployed across areas unless there are apparent exceptions.

The storage resources and bandwidth are all capacitated. The storage resources of microservice for applications affect the hit rate of requested contents, and the bandwidth resources affect the transmission latency. If too many storage and bandwidth

resources are allocated, the performance outdistances the applications needed. This means the deployment cost is more expensive and causes insufficient resources. To avoid the high deployment cost, we define a latency range that describes the application's tolerance. Therefore, the goal of this paper is the problem of minimizing the total deployment bandwidth and storage resource cost within the application's tolerance latency. The problem formulation will be introduced in the next section.

4 | PROBLEM FORMULATION

The primary objective of this paper is to identify the set H , which encompasses the subset of ESs that function specifically as cache servers among all ESs within the system. The system of this paper consists of two parts: $\mathbb{S} = \{s_1, s_2, \dots, s_k\}$ as all ESs set in one region and $\mathbb{A} = \{a_1, a_2, \dots, a_m\}$ as the set of all Apps. Bit s_i identifies whether ES i is a deployed one: $s_i = 1$ for ES $i \in H$, and $s_i = 0$ for ES $i \notin H$. To optimize the deployment of an ES, the latency of client users obtaining request contents must be taken into consideration, which is affected by storage and bandwidth allocation for ESs. Therefore, the system of this paper considers two types of resources: storage and bandwidth. SS_{s_i} and SB_{s_i} present the total storage and bandwidth capacity of the server s_i , respectively. Per-ES's storage and bandwidth resources are available to each application. It means that each application can be allocated the storage and bandwidth resources from different servers. ss_{s_i, a_j} and sb_{s_i, a_j} represent the storage resources and bandwidth resources allocated in application a_j on server s_i , where $ss_{s_i, a_j} \leq SS_{s_i}$, $sb_{s_i, a_j} \leq SB_{s_i}$, $s_i \in \mathbb{S}$ and $a_j \in \mathbb{A}$. The storage $ST = [ss_{s_1, a_1}, ss_{s_1, a_2}, \dots, ss_{s_k, a_m}]$ and bandwidth $BW = [sb_{s_1, a_1}, sb_{s_1, a_2}, \dots, sb_{s_k, a_m}]$ resources allocated for each application are the other outputs.

The requested content has different sizes, affecting the transmission latency, which includes client-server latency lu and server-cloud latency lc . L_{a_j} presents the latency generated in application a_j and is computed by

$$L_{a_j} = lu_{a_j} + lc_{a_j}. \quad (1)$$

To ensure QoS of users, each application a_j is set the tolerance latency range $[\sigma_{a_j}, \omega_{a_j}]$. So, the L_{a_j} should in $[\sigma_{a_j}, \omega_{a_j}]$. We adopt n_{a_j} to imply the number of request content in application a_j . N implies the total number of request content in all applications. If the requested content is uncached in ESs, the ES needs to pull the missing content from the cloud. This paper assumes the bandwidth between ES and cloud as ξ . The lu_{a_j} is computed by

$$lu_{a_j} = \frac{\sum_{q=1}^{n_{a_j}} rs_{q, a_j}}{\sum_{i=1}^m sb_{s_i, a_j}}. \quad (2)$$

rs_{q, a_j} presents the size of the request number q in application a_j . The lc_{a_j} presents the transmission latency for fetching *miss*

contents and is computed by

$$lc_{a_j} = (1 - br_{a_j}) \frac{\sum_{q=1}^{n_{a_j}} rs_{q, a_j}}{\xi}. \quad (3)$$

The br_{a_j} is the hit rate of application a_j . The hit rate is affected by storage resources. In the same cache replacement algorithm, the larger the caching space size, the higher the hit rate. To find the relationship between storage and hit rate, we used the LRU algorithm with actual data from a CDN company *ChuangCache* and recorded the change in hit rate by changing the cache size. It is found that the caching space size (storage allocated) and the hit rate satisfy a positive correlation akin to a one-to-one linear relationship based on experiments. The paper approaches it as a direct one-to-one linear relationship ($y = a * x + b$). The connection is fitted by the data to calculate a as well as b . Finally, the latency lc_{a_j} is calculated as follows.

$$lc_{a_j} = \left(1 - a_{a_j} * \sum_{q=1}^{n_{a_j}} ss_{s_i, a_j}^q - b_{a_j} \right) \frac{\sum_{q=1}^{n_{a_j}} rs_{q, a_j}}{\xi}. \quad (4)$$

ss_{s_i, a_j}^q presents the storage resources allocated for the q th request in application a_j on server s_i . The objective of this paper is to minimize the system's total cost C_{total} in the hybrid deployment of ESs. The cost includes three sub-costs: bandwidth-cost C_{bw} , storage-cost C_{st} , and server-cost C_{server} . The total cost is calculated as follows:

$$C_{\text{total}} = \alpha * C_{\text{bw}} + \beta * C_{\text{st}} + \gamma * C_{\text{server}}. \quad (5)$$

α, β, γ are the weight factors to balance the objectives between bandwidth cost, storage cost, and server cost.

The bandwidth cost is the total bandwidth allocated in all ESs for all applications and is computed by

$$C_{\text{bw}} = \sum_{i=1}^k \sum_{j=1}^m sb_{s_i, a_j}. \quad (6)$$

The storage cost is the total storage allocated in all servers for all applications and is computed by

$$C_{\text{st}} = \sum_{i=1}^k \sum_{j=1}^m ss_{s_i, a_j}. \quad (7)$$

As we stated before, system costs are mainly composed of three parts: bandwidth cost, storage cost, and server cost. For the first two, we calculate the cost by adding up the resources used. But for the server cost, if we use a dichotomous variable to indicate whether or not resources are allocated on the server s_i , the cost formula becomes non-linear. This approach indirectly shows the exact cost of the server. To solve this problem, we measure the server cost of each server in terms of the

TABLE 1 Main notations.

Notation	Semantics
$\mathbb{S}$	Set of all ESs
$\mathbb{A}$	Set of all Apps
H	Set of deployed servers across $\mathbb{S}$
s_i	Bit indicating whether deployed server
SS_{s_i}	Storage capacity of server s_i
SB_{s_i}	Bandwidth capacity of server s_i
ss_{s_i,a_j}	Storage resources allocated in application i on server j
sb_{s_i,a_j}	Bandwidth resources allocated in application i on server j
lu	Client-server latency
lu_{a_j}	Client-server latency in application a_j
lc	Server-cloud latency
lc_{a_j}	Server-cloud latency in application a_j
rs_{a_j}	Request size in application a_j
ξ	Bandwidth between ES with cloud
C_{bw}	Bandwidth-cost
C_{st}	Storage-cost
C_{server}	Server-cost
C_{total}	Total cost
α, β, γ	Cost-function factors

proportion of resources allocated to a server, thus transforming the non-linear problem into a linear one. The server cost is the total resources allocated proportion in all servers for all applications and is computed by

$$C_{server} = \sum_{i=1}^k \left(\frac{\sum_{j=1}^m ss_{s_i,a_j}}{SS_{s_i}} + \frac{\sum_{j=1}^m sb_{s_i,a_j}}{SB_{s_i}} \right). \quad (8)$$

Based on the above statements, the problem to be addressed in this section can be represented by the following mathematical model.

$$\min \sum_{i=1}^k \sum_{j=1}^m C_{total}^{i,j} \quad (9)$$

$$\text{s.t.} \quad 0 \leq \sum_{i=1}^k sb_{s_i,a_j} \times s_i \leq SB_{s_i} \quad (10)$$

$$0 \leq \sum_{i=1}^k ss_{s_i,a_j} \times s_i \leq SS_{s_i} \quad (11)$$

$$\sigma_{a_j} \leq L_{a_j} \leq \omega_{a_j}. \quad (12)$$

With all the notation summarized in Table 1, we formulate the problem of this paper as follows:

- Inputs: ESs set $\mathbb{S}$, Applications set $\mathbb{A}$, tolerance latency range $[\sigma_{a_j}, \omega_{a_j}]$, cost function C_{total} ;
- Objective: Minimize cost C_{total} ;

- Constraints: Equations (10)–(12).
- Outputs: ESs deployed set H , the bandwidth BW and storage ST resources allocated for each application.

5 | SOLUTION

Based on the problem formulation, the deployment of ESs for multi-application is an MIP problem. We use the Gurobi optimizer, a popular and widely used commercial optimization solver to solve this problem. We ensure that the maximum optimality gap is 10%. Due to the time-varying nature of requests, the resources of each application are different in two consecutive intervals. It is necessary for the dynamic and elastic allocation of resources in each application to reduce the cost and ensure the performance of ESs. Performing resource allocation in real-time is unwise, which not only creates redundant transfers but may increase user response time. Therefore, it seems impossible to resolve the MIP problem by entering all the input values every moment. To resolve this problem, we propose a solution *HybridDep* and divide it into two phases: coarse-grained scheduling and fine-grained tuning.

5.1 | Coarse-grained

For coarse-grained purposes, we divide the day into 24-h zones. Because such data essentially originates from human behaviour, which often exhibits a daily periodic pattern [27, 28]. To make the best use of resources, we deploy applications geographically based on the characteristics of the peak and valley staggered in the same region. At the beginning of each time zone, the system calculates the average number of requests per hour for that time zone based on the previous day. Then these values are entered into the system with other needed input values. After obtaining the resource allocation *Optimal* for each time zone through coarse-grained deployment, the system performs fine-grained tuning in each time zone.

5.2 | Fine-grained tuning

The goal of fine-grained tuning is to meet the application's latency requirements. As shown in Equation (4), the latency is related to storage and bandwidth resources. As mentioned above, to better characterize the latency formula, we find the unary relation between hit rate and cache size through experiments. The request interval is varied both in different periods and in different applications. If fine-grained tuning is performed every moment, the system will be adjusted frequently, bringing unnecessary overhead. To get closer to real-time tuning, we choose the suitable access interval per application as the time interval *timeSlot*, in which each application performs fine-grained tuning. The reason for this is that for some requests with a small number of requests, there will be no requests in a short period. If *timeSlot* is set to a fixed time, in some high-frequency applications, latency often exceeds the tolerated

latency range for a long time before being noticed, which leads to low QoS.

After each application a_j experiences its time interval, the system calculates the average request size per interval. It then calculates the application's latency according to the latency formula and resource allocation. The next time interval is entered if the latency is less than or equal to the upper limit of latency or greater than or equal to the lower limit of latency. Otherwise, if the real latency $l_{old}^{a_j}$ is greater than or equal to the upper limit of latency, the upper limit of latency is set as the target latency $l_{new}^{a_j}$. If the real latency is less than or equal to the lower latency limit σ_{a_j} , the upper latency limit ω_{a_j} is set to the target latency. We then list the formulas for the real and target latency below:

$$l_{new}^{a_j} = \sum_{q=1}^{n_{a_j}} rs_{q,a_j} \times \left(\frac{1}{\sum_{i=1}^k sb_{s_i,a_j}} + \frac{1 - bitrate_{new}}{\xi} \right), \quad (13)$$

$$l_{old}^{a_j} = \sum_{q=1}^{n_{a_j}} rs_{q,a_j} \times \left(\frac{1}{\sum_{i=1}^k sb'_{s_i,a_j}} + \frac{1 - bitrate_{old}}{\xi} \right). \quad (14)$$

The amount of requests in these two formulas is the same. So, we subtract these two formulas to get the difference between the target resource allocation and the actual resource allocation for fine-grained tuning. However, there are two unknowns in this difference formula, namely, bandwidth allocation difference and storage allocation difference. To properly allocate storage and bandwidth resources, and transform two unknowns into ones, we define the ratio of bandwidth adjustment to storage adjustment as $a : b$. This ratio is determined by the bandwidth resources and storage resources of all ESs in the region. Therefore, the difference between the above two expressions can be shown as follows:

$$\delta latency_{a_j} = l_{old}^{a_j} - l_{new}^{a_j} \quad (15)$$

Combining the Equations (13) and (14), the $\delta latency$ is computed by

$$\delta latency_{a_j} = \sum_{q=1}^{n_{a_j}} rs_{q,a_j} \times \left(\frac{sb_{a_j} - sb'_{a_j}}{sb_{a_j} \times sb'_{a_j}} + \frac{ss_{a_j} - ss'_{a_j}}{\xi} \right) \quad (16)$$

Then, we simplify it to get

$$\frac{\delta latency_{a_j}}{\sum_{q=1}^{n_{a_j}} rs_{q,a_j}} = \frac{\delta sb_{a_j}}{sb_{a_j}^2 + sb'_{a_j} \times \delta sb_{a_j}} + \frac{a \times \delta sb_{a_j}}{b \times \xi}, \quad (17)$$

where sb_{a_j} and sb'_{a_j} respectively present equation as follows:

$$sb_{a_j} = \sum_{i=1}^k sb_{s_i,a_j}, sb'_{a_j} = \sum_{i=1}^k sb'_{s_i,a_j}. \quad (18)$$

We define the $\delta sb_{a_j} = x$ and gain the equation:

$$\frac{\delta latency_{a_j}}{\sum_{q=1}^{n_{a_j}} rs_{q,a_j}} = \frac{x}{sb_{a_j}^2 + sb'_{a_j} \times x} + \frac{b \times x}{a \times \xi}. \quad (19)$$

After the above derivation, we can compute x by Equation (20).

$$0 = \frac{b \times sb'_{a_j}}{a \times \xi} \times x^2 + \left(\frac{b \times sb_{a_j}^2 \times w_i}{a} - sb'_{a_j} \times \xi + \xi \right) \times x - sb_{a_j}^2 \times \xi \times \frac{\delta latency_{a_j}}{\sum_{q=1}^{n_{a_j}} rs_{q,a_j}} \quad (20)$$

The increasing or decreasing resource allocated can be calculated to meet the target latency by solving the above bivariate equation. While it is possible to get the increasing (decreasing) storage and bandwidth resources, the choice of which servers to add and decrease remains an issue. This is because it is directly related to the overall cost of the server. To solve this problem, we propose different solutions for adding and reducing resources, as shown below.

To avoid the waste of resources when deploying new applications, which generate extra system files, the strategy should deploy as few applications as possible in a server. If resources of an application a_i need to be added, the system arranges ESs in ascending order of remaining resources, which already allocated resources for application a_i . Then, coarse-grained allocates additional resources preferentially to ESs with fewer remaining resources. The goal is to fill as many ESs as possible and reduce external fragmentation. On the contrary, when reducing the resources, the system will apply the resources on different ESs in ascending order. The priority is to remove the reduced resources from the ESs with fewer resources. The purpose of this step is to enable the system to free up spare ESs

5.3 | System framework

Combining the two phases: coarse-grained scheduling and fine-grained tuning, we present the system framework of *HybridDep*. The framework mainly includes three parts: The bandwidth and storage resources of requests in real time, regular coarse-grained scheduling, and conditionally triggered fine-grained tuning, shown in Figure 3.

- Obtaining resources of request in real time: The system collects request frequency and content size in real time, which provides data support for calculating the required storage and bandwidth resources based on the current access volume and the tolerable latency range.
- Regular coarse-grained scheduling: In this system, we set a fixed time interval T to reallocate the resource requirements

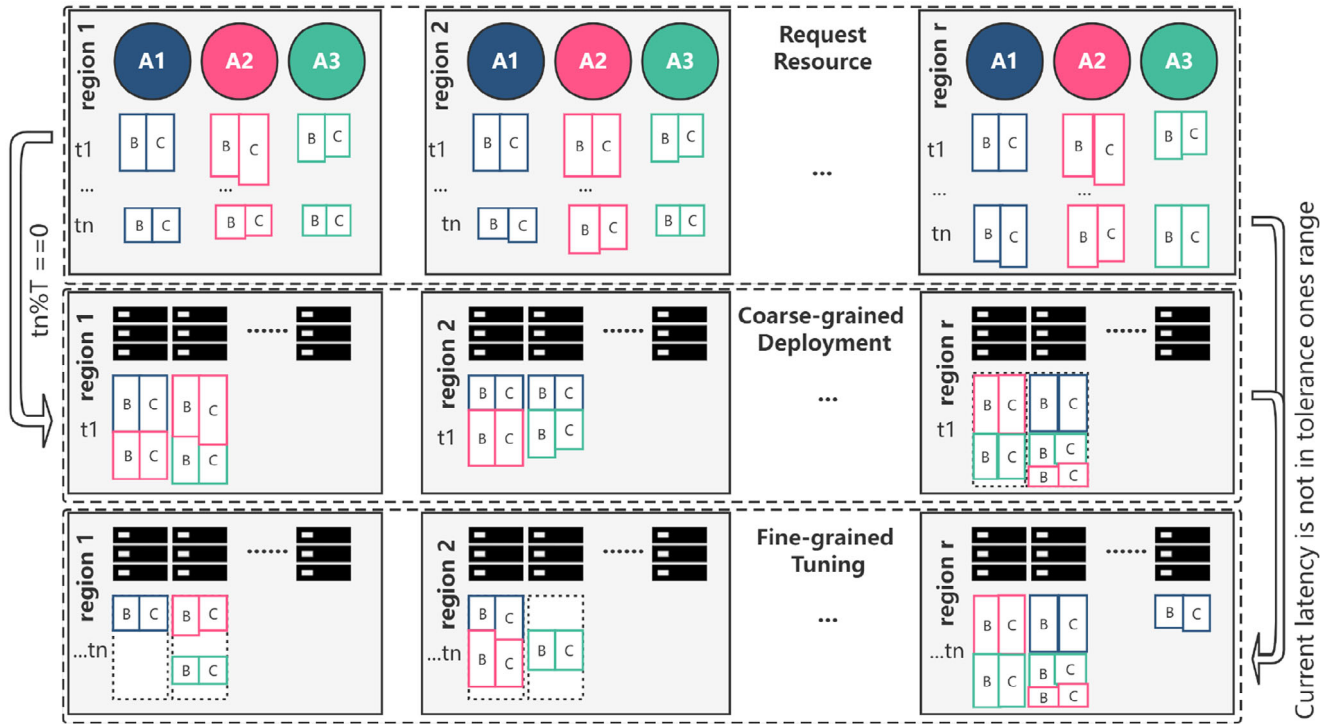


FIGURE 3 The framework overview.

for all applications. Once this time is up, we perform coarse-grained scheduling, assigning resources to each ES for each application. (See in lines 2–5 of the Algorithm 1)

- **Conditionally triggered fine-grained tuning:** A QoS monitoring of the application is performed by setting a fixed period to calculate the request latency of the application. Once the request latency is without in the preset latency tolerance range, it means that too many or too few resources are allocated. It is then necessary to perform fine-grained turning, corresponding reduction, and expansion operations. This enables maintaining the application's QoS through further tiny adjustments (See in lines 6–20 of the Algorithm 1). The theoretical time complexity of the algorithm is $O(n \log n)$.

The pseudo-code of the algorithm *HybridDep* is shown in Algorithm 1.

6 | EVALUATION

In this section, we evaluate our approach *HybridDep* by real traces from *ChuangCache*, show the results of applying *HybridDep* and other algorithms, and analyse the results of all comparing algorithms.

6.1 | Dataset

The dataset used in the paper is sourced from a CDN company *ChuangCache* in one region of China and is shown in Table 2. The dataset concludes three real traces of I/O-intensive appli-

cations. The total request frequencies for the three traces are 29,069, 5,366,261, and 28,816,961, respectively. We have classified traces into three categories based on the corresponding request frequency: Low, Medium, and High. The three traces' average inter-request time (AIRT) are 2972318, 16.11 and 3.00 *ms*, respectively.

6.2 | Comparing algorithms

- **Optimal:** Assuming precise foresight into future resource demands, *Optimal* builds upon the advanced methodology detailed in paper [13], and seamlessly integrates the peak-valley interleaving discussed herein, specific to the same geographical region. This fusion enables dynamic adjustment of resource allocation strategies, tailored to the cyclical fluctuations in demand, thereby striving for optimal resource allocation.
- **Coarse-Grained:** To validate the effectiveness and practicality of *HybridDep*, given the inability to predict the precise amount of resource demands, we undertook a comparative experiment between *HybridDep* and an optimal algorithm, *Optimal*, which hypothetically possesses the capability of acquiring precise future resource demands. Since the optimal algorithm here refers to a one-time, fixed resource deployment scheme based on complete prior knowledge of each application's resource demand in the future, it aims to demonstrate that these traditional load balancing methods (i.e. peak shaving and valley filling) are not applicable in application scenarios with high resources demand variability. Within this framework, we utilized the *Coarse-Grained* scheduling outcome as an

ALGORITHM 1 The pseudo-code of *HybridDep*

Input: ESs set $\mathcal{S}$; Applications set $\mathcal{A}$; Tolerance latency range $[\sigma_{a_j}, \omega_{a_j}]$; Cost function C_{total} ;

Output: ESs deployed set H ; The bandwidth BW and storage ST resources allocated for each application.

```

1 while true do
2   if timeStamp%T == 0 then
3     compute  $BW, ST$  by coarse-grained scheduling;
4     allocates resources to each ES according to  $BW, ST$ ;
5     update ESs deployed set  $H$ ;
6   end
7   else
8     for  $j$  to  $m$  do
9       if timeStamp%timeSlot[j] == 0 then
10        compute the real latency  $l_{old}^{a_j}$  by equation (14);
11        if  $l_{old}^{a_j} > \omega_{a_j}$  or  $l_{old}^{a_j} < \sigma_{a_j}$  then
12          compute  $sb_{a_j}, ss_{a_j}$ ;
13          allocate resource to each ES according to  $sb_{a_j}, ss_{a_j}$ ;
14        end
15        else
16          continue;
17        end
18        calculate cost  $C_{total}^{a_j}$ ;
19        update ESs deployed set  $H$ ;
20      end
21    end
22  end
23 end

```

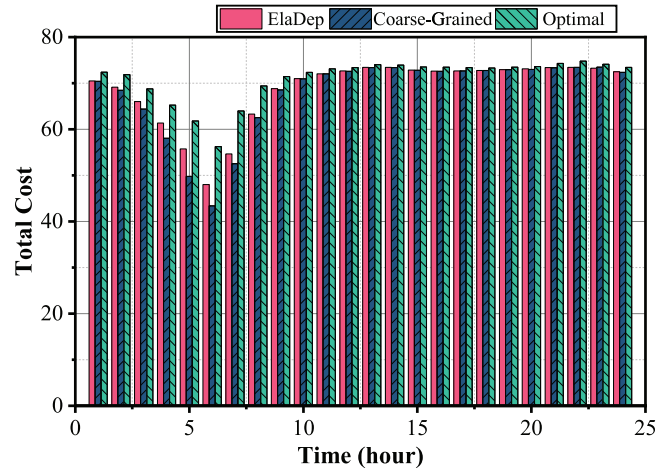
TABLE 2 Dataset.

App item	Total request	Total time	AIRT
Low	29,069	24 h	2972.18 ms
Medium	5,366,261	24 h	16.11 ms
High	28,816,961	24 h	3.00 ms

intermediary step, aiming to substantiate the imperative need for elastic resource allocation, also known as fine-grained tuning. This approach enabled us to demonstrate how *HybridDep*, through its adaptability and precision in resource management, surpasses simpler strategies that lack the capability for nuanced adjustments.

6.3 | Performance comparison

Our method *HybridDep* for solving the *ElaDep* problem splits two phases: coarse-grained scheduling and fine-grained tuning. We quantify how accurately the storage and bandwidth

**FIGURE 4** The total cost of three applications.

resources allocated represent the total costs for each ES under the tolerance latency. Our method *HybridDep* elastically allocates resources by increasing and decreasing the required resources with time change for multi-application. This paper chooses three algorithms: *Optimal*, *Coarse-Grained*, and *HybridDep* to analyse applications' total cost and latency comparatively.

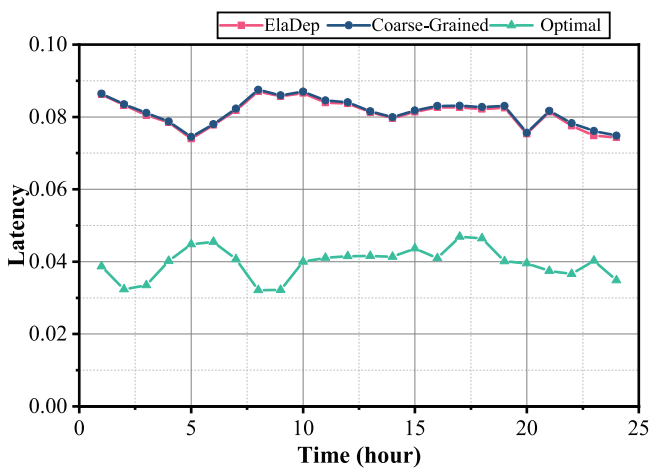
Figure 4 represents the 'total cost' of deployment for three applications. The total cost can be reduced by 3.2% by using *HybridDep* to deploy three applications compared to *Optimal* in 1 day. In particular, at hour 6, the cost can be reduced by 14.6% by using *HybridDep* than *Optimal*. In addition, we can find that the *Optimal* (green bar) is higher than other algorithms in each hour, especially in hours 3–7. The total cost per hour of *Coarse-Grained* is almost lower than others. This is because *Coarse-Grained* statically allocates resources based on historical request data from the previous day. This means that it ignores the time-varying nature of the resources required, which can result in exceeding the latency tolerance range. To verify that the phenomenon exists, we show the latency of applications from two perspectives: the average latency and the time of over-tolerance latency.

This paper aims to satisfy the application QoS while minimizing the deployment cost, so it is necessary to show the **latency** metric that directly affects the QoS. Figure 5 plots the average latency of three applications in 24 h. Due to the over-allocated resources in *Optimal*, the latency is obviously lower than others. *HybridDep* is slightly lower in latency than *Coarse-Grained* because the former allocates more resources than the latter to prevent application latency from exceeding the tolerated latency and thus affecting applications' QoS. This means that our algorithm sacrifices some of the cost to ensure the QoS of applications.

By monitoring the latency, *HybridDep* can maintain a high QoS for the application. It is guaranteed that the latency of all applications will not exceed the tolerance latency shown in Table 3. Table 3 counts the total minutes of over-tolerance latency in 24 h in about three applications. It is easy to see that the excess time per hour at *Optimal* is zero, both in low, medium, and high frequency applications. Especially, *HybridDep* has the same result with *Optimal*. It means that our algorithm

TABLE 3 The minutes of over tolerate latency in three applications.

App Item	Algorithms	Time (h)																									
		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	total	
Low	HybridDep	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Coarse-Grained	3	2	2	1	0	1	2	2	3	4	4	3	2	3	2	2	2	2	2	1	1	0	1	2	47	
	Optimal	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
Medium	HybridDep	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Coarse-Grained	1	1	1	1	2	1	2	2	1	2	1	1	1	1	1	1	3	1	2	1	1	1	1	1	31	
	Optimal	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
High	HybridDep	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Coarse-Grained	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
	Optimal	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

**FIGURE 5** The average latency of three applications.

HybridDep achieves the same QoS with low-cost deployments compared with algorithm *Optimal*, which has high-cost deployment. Although the deployment cost of *Coarse-Grained* is lower than *HybridDep*, the total minutes of over-tolerance latency is bigger than zero almost every hour. In other words, *Coarse-Grained* is unable to ensure the QoS of applications. Therefore, the fine-grained tuning phase of *HybridDep* in this paper is a necessary design for keeping a high QoS for applications.

6.4 | Result analysis

Since each application is requested at different frequencies, the ‘total cost’ of deployment varies from application to application. Figure 6 includes three sub-figures (Figure 6a–c), which shows the result of deployment cost for three applications. We can find that the deployment cost of our algorithm *HybridDep* (red bar) is lower than *Optimal* (green bar) in either application, especially in low frequency application (Figure 6a). After analysis, the reason for the above phenomenon is that the request frequency varies significantly in low frequency application. In contrast, the request frequency is relatively stable in medium frequency appli-

cation (Figure 6b), so the cost of deployment is relatively stable. This also means that our algorithm *HybridDep* is more effective in applications or periods where the frequency of requests is more time-varying, such as low frequency one and hours 2–7 in high frequency one (Figure 6c).

Figure 7 shows the average latency of each hour with three algorithms in three applications about 24 h. The average latency of our algorithm *HybridDep* (red line) is in the middle of the range, both in low, medium, and high frequency applications. *HybridDep*’s latency is higher than *Optimal*’s, but it is within the tolerance latency range. In particular, in low frequency application, the latency of *Optimal* differs significantly with other algorithms from other applications. This means that there is a significant potential for improvement in deployment costs in low frequency application. This view is also proved by the total cost result of low frequency application in Figure 6a.

In summary, our algorithm *HybridDep* can achieve the elastic hybrid resources allocation for multi-application of I/O-intensive by adopting the peak clipping and valley filling in the same region, subject to the latency constraints, reducing total cost improvement over *Optimal*. We believe that our approach is suitable for other applications that exhibit similar characteristics and issues.

7 | CONCLUSION

This paper finds that it is challenging to elastically allocate the storage and bandwidth resources by adopting the hybrid deployment strategy, due to the time-varying and continuity of request frequency. By analysing the real data, we find that peak clipping and valley filling are not easily implemented by directly adopting hybrid multi-application deployment based on temporal dimension. It is because that the time of peaks and valleys appeared are similar for each application. To solve this challenge, we deeply analyse the real data and traces of multiple I/O-intensive applications from a CDN company *ChuangCache* in China and find that the peak and valley of different applications are staggered in the same region. On this basis, we propose an elastic hybrid of the bandwidth and

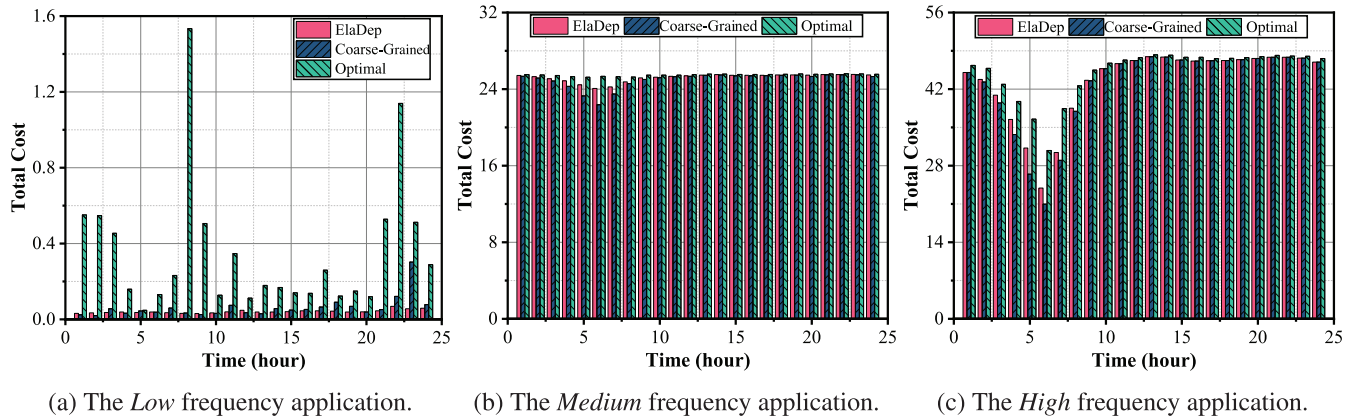


FIGURE 6 (Result analysis) The total cost of allocation resources in three applications.

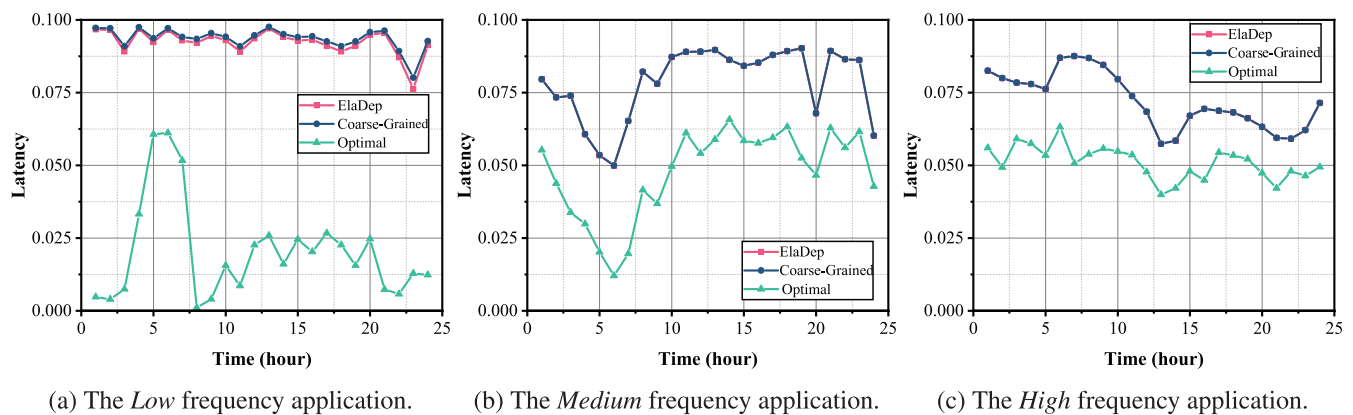


FIGURE 7 (Result analysis) The average latency of three applications.

storage resources allocation strategy *HybridDep* for I/O-intensive applications to minimize the total cost while keeping a high applications' QoS. *HybridDep* includes two phases: coarse-grained scheduling and fine-grained tuning. Through a series of experimental verifications, our strategy is better than *Optimal* in deployment cost under the tolerance latency range.

AUTHOR CONTRIBUTIONS

Pengmiao Li: Writing—original draft. **Yuchao Zhang:** Writing—review and editing. **Shaoxuan Yun:** Validation. **Fuca Yu:** Funding acquisition. **Aizhi Wu:** Funding acquisition.

ACKNOWLEDGEMENTS

This work was supported in part by the Beijing Nova Program under Grant 2023140, the National Natural Science Foundation of China under Grant 62172054, and the Beijing Science and Technology Planning Project (Z231100003823005).

CONFLICT OF INTEREST STATEMENT

The authors declare no conflicts of interest.

DATA AVAILABILITY STATEMENT

The data will be made available on request due to privacy and ethical restrictions.

ORCID

Pengmiao Li  <https://orcid.org/0000-0002-6823-2161>

REFERENCES

1. Cisco: White Paper 2022. <https://www.cisco.com> (2023). Accessed June 2023
2. Social&Hootsuite, W.A.: Global Overview Report. <https://wearesocial.cn/> (2023). Accessed June 2023
3. Gu, L., Chen, Z., Xu, H., Zeng, D., Li, B., Jin, H.: Layer-aware collaborative microservice deployment toward maximal edge throughput. In: Proceedings of the IEEE INFOCOM 2022—IEEE Conference on Computer Communications, pp. 71–79. IEEE, Piscataway, NJ (2022). <https://doi.org/10.1109/INFOCOM48880.2022.9796670>
4. Samanta, A., Tang, J.: Dyme: Dynamic microservice scheduling in edge computing enabled IOT. IEEE Internet Things J. 7(7), 6164–6174 (2020). <https://doi.org/10.1109/JIOT.2020.2981958>
5. Herbaut, N., Négru, D., Dietrich, D., Papadimitriou, P.: Dynamic deployment and optimization of virtual content delivery networks. IEEE Multimedia 24(3), 28–37 (2017). <https://doi.org/10.1109/MMUL.2017.3051510>
6. Kamiyama, N., Hosokawa, Y.: Optimally designing virtualized CDN maximizing profit of content providers. In: Proceedings of the 16th IEEE Annual Consumer Communications and Networking Conference, CCNC 2019, pp. 1–4. IEEE, Piscataway, NJ (2019). <https://doi.org/10.1109/CCNC.2019.8651800>
7. Akkus, I.E., Chen, R., Rimac, I., Stein, M., Satzke, K., Beck, A., Aditya, P., Hilt, V.: SAND: Towards high-performance serverless computing. In:

- Gunawi, H.S., Reed, B. (eds.) Proceedings of the 2018 USENIX Annual Technical Conference, USENIX ATC 2018, pp. 923–935. USENIX Association, Boston, MA, USA (2018). <https://www.usenix.org/conference/atc18/presentation/akkus>
8. Oubaydallah, Y., Ibrahim, K., Azouzi, R.E.: Recent advances in data intensive applications: Survey. In: Ibrahim, K., El-Kamili, M., Kobbane, A., Shayea, I. (eds.) Proceedings of the 10th International Conference on Wireless Networks and Mobile Communications, WINCOM 2023, pp. 1–6. IEEE, Piscataway, NJ (2023). <https://doi.org/10.1109/WINCOM59760.2023.10322920>
 9. Tang, C., Wang, Y., Fan, B., Wang, B., Chen, S., Qiu, Z., Liang, C., Zhao, J., Zhu, Y., Chen, M., Hu, Z.: Rethinking the cloudonomics of efficient I/O for data-intensive analytics applications. arXiv:2311.00156 (2023). <https://doi.org/10.48550/ARXIV.2311.001562311.00156>
 10. Toumi, H., Brahmi, Z., Gammoudi, M.M.: RTSLPS: Real time server load prediction system for the ever-changing cloud computing environment. J. King Saud Univ. Comput. Inf. Sci. 34(2), 342–353 (2022). <https://doi.org/10.1016/j.jksuci.2019.12.004>
 11. Bayram, F., Aupke, P., Ahmed, B.S., Kassler, A., Theocharis, A., Forsman, J.: DA-LSTM: A dynamic drift-adaptive learning framework for interval load forecasting with LSTM networks. Eng. Appl. Artif. Intell. 123(PC), 106480 (2023). <https://doi.org/10.1016/j.engappai.2023.106480>
 12. Li, P., Zhang, Y., Wang, W., Zhao, K., Lian, B., Xu, K., Zhang, Z.: Fend for edge servers: Reduce server number! Keeping service quality! In: Proceedings of the 2021 IEEE 23rd International Conference on High Performance Computing and Communications, pp. 107–114. IEEE, Piscataway, NJ (2021). <https://doi.org/10.1109/HPCC-DSS-SmartCity-DependSys53884.2021.00041>
 13. Hasan, S., Gorinsky, S., Dovrolis, C., Sitaraman, R.K.: Trade-offs in optimizing the cache deployments of cdns. In: Proceedings of the 2014 IEEE Conference on Computer Communications, INFOCOM 2014, pp. 460–468. IEEE, Piscataway, NJ (2014). <https://doi.org/10.1109/INFOCOM.2014.6847969>
 14. Lien, S., Hung, S., Hsu, H., Deng, D.: Energy-optimal edge content cache and dissemination: Designs for practical network deployment. IEEE Commun. Mag. 56(5), 88–93 (2018). <https://doi.org/10.1109/MCOM.2018.1700891>
 15. Zhang, S., Zhang, N., Yang, P., Shen, X.: Cost-effective cache deployment in mobile heterogeneous networks. IEEE Trans. Veh. Technol. 66(12), 11264–11276 (2017). <https://doi.org/10.1109/TVT.2017.2724547>
 16. Ohashi, T.: Analysis of QOS in heterogeneous networks with clustered deployment and caching aware capacity allocation. arXiv:2103.12993 (2021). <https://arxiv.org/abs/2103.12993>
 17. Gourdin, É., Maillé, P., Simon, G., Tuffin, B.: The economics of CDNS and their impact on service fairness. IEEE Trans. Network Serv. Manage. 14(1), 22–33 (2017). <https://doi.org/10.1109/TNSM.2017.2649045>
 18. Harter, T., Salmon, B., Liu, R., Arpaci-Dusseau, A.C., Arpaci-Dusseau, R.H.: Slacker: Fast distribution with lazy docker containers. In: Brown, A.D., Popovici, F.I. (eds.) Proceedings of the 14th USENIX Conference on File and Storage Technologies, FAST 2016, pp. 181–195. USENIX Association, Santa Clara, CA (2016). <https://www.usenix.org/conference/fast16/technical-sessions/presentation/harter>
 19. Du, L., Wo, T., Yang, R., Hu, C.: CIDER: A rapid docker container deployment system through sharing network storage. In: Proceedings of the 19th IEEE International Conference on High Performance Computing and Communications; 15th IEEE International Conference on Smart City; 3rd IEEE International Conference on Data Science and Systems, pp. 332–339. IEEE Computer Society, Washington, D.C. (2017). <https://doi.org/10.1109/HPCC-SmartCity-DSS.2017.44>
 20. Gao, B., Zhou, Z., Liu, F., Xu, F.: Winning at the starting line: Joint network selection and service placement for mobile edge computing. In: 2019 IEEE Conference on Computer Communications, INFOCOM 2019, pp. 1459–1467. IEEE, Piscataway, NJ (2019). <https://doi.org/10.1109/INFOCOM.2019.8737543>
 21. Wang, L., Jiao, L., He, T., Li, J., Mühlhäuser, M.: Service entity placement for social virtual reality applications in edge computing. In: Proceedings of the 2018 IEEE Conference on Computer Communications, INFOCOM 2018, pp. 468–476. IEEE, Piscataway, NJ (2018). <https://doi.org/10.1109/INFOCOM.2018.8486411>
 22. Ramalho, F., Neto, A.: Virtualization at the network edge: A performance comparison. In: 17th IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks, WoWMoM 2016, pp. 1–6. IEEE Computer Society, Washington, D.C. (2016). <https://doi.org/10.1109/WoWMoM.2016.7523584>
 23. Gu, L., Zeng, D., Hu, J., Jin, H., Guo, S., Zomaya, A.Y.: Exploring layered container structure for cost efficient microservice deployment. In: Proceedings of the 40th IEEE Conference on Computer Communications, INFOCOM 2021, pp. 1–9. IEEE, Piscataway, NJ (2021). <https://doi.org/10.1109/INFOCOM42981.2021.9488918>
 24. Gu, L., Zeng, D., Hu, J., Li, B., Jin, H.: Layer aware microservice placement and request scheduling at the edge. In: Proceedings of the 40th IEEE Conference on Computer Communications, INFOCOM, pp. 1–9. IEEE, Piscataway, NJ (2021). <https://doi.org/10.1109/INFOCOM42981.2021.9488779>
 25. Poularakis, K., Llorca, J., Tulino, A.M., Taylor, I.J., Tassiulas, L.: Joint service placement and request routing in multi-cell mobile edge computing networks. In: Proceedings of the 2019 IEEE Conference on Computer Communications, INFOCOM 2019, pp. 10–18. IEEE, Piscataway, NJ (2019). <https://doi.org/10.1109/INFOCOM.2019.8737385>
 26. Ferreira, D.L., Nunes, B.A.A., Campos, C.A.V., Obraczka, K.: A deep learning approach for identifying user communities based on geographical preferences and its applications to urban and environmental planning. ACM Trans. Spatial Algorithms Syst. 6(3), 17–11724 (2020). <https://doi.org/10.1145/3380970>
 27. Zhang, Y., Li, P., Zhang, Z., Bai, B., Zhang, G., Wang, W., Lian, B.: Challenges and chances for the emerging short video network. In: Proceedings of the IEEE INFOCOM 2019—IEEE Conference on Computer Communications Workshops, INFOCOM Workshops 2019, pp. 1025–1026. IEEE, Piscataway, NJ. <https://doi.org/10.1109/INFCOMW.2019.8845192>
 28. Jung, G., Park, S., Ma, E., Kim, H., Lee, U.: Tutorial on matching-based causal analysis of human behaviors using smartphone sensor data. ACM Comput. Surv. 56(9), 236–123633 (2024). <https://doi.org/10.1145/3648356>

How to cite this article: Li, P., Zhang, Y., Yun, S., Yu, F., Wu, A.: *HybridDep*: An elastic hybrid resources allocation strategy for I/O-intensive applications. IET Commun. 19, e70007 (2025). <https://doi.org/10.1049/cmu2.70007>